Q1. What is a NoSQL database? Briefly describe the four main types of NoSQL databases, giving one real-world example for each type.

Q2. Distinguish between a document store and a key-value store. Provide one real-world use case where each is the most appropriate choice.

Q3. What is a column-family (wide-column) database? Explain how it differs from a traditional RDBMS column and name one widely used column-family database.

Q4. State any four advantages of NoSQL databases over traditional relational databases.

Q5. What is horizontal scalability? Explain why NoSQL databases can achieve it more easily than SQL databases.

Q6. What does the CAP theorem state? Using the CAP triangle, indicate where MongoDB, Cassandra, and Redis each fall, and what guarantees each sacrifices.

Q7. What is eventual consistency in NoSQL? Contrast it with strong consistency and give a scenario where eventual consistency is acceptable.

Q8. Expand BASE in the context of NoSQL systems. For each property, write the equivalent ACID property and explain how they differ.

Q9. Explain the concept of sharding in NoSQL databases. A document store holds 800 million records distributed equally across 8 shards. (i) How many records are on each shard? (ii) How many shards must a range query scan if it targets 12.5% of all keys?

Q10. When would you prefer a graph database over a document database? Describe a recommendation-engine use case to support your answer.

Q11. Map the following RDBMS terms to their MongoDB equivalents: Database, Table, Row, Column, Primary Key, Join.

Q12. List any four reasons why MongoDB is preferred over a traditional RDBMS for modern web applications.

Q13. What is a BSON document? How does it differ from a plain JSON document? Provide a sample BSON document representing a library book record.

Q14. List at least six BSON data types supported by MongoDB. Give one brief example value for each.

Q15. Write a MongoDB query to find all documents in an 'employees' collection where the 'salary' field is greater than 60000 and the 'department' is 'Engineering'.

Q16. Write MongoDB queries to: (i) update the salary of employee 'Ravi Kumar' to 75000, and (ii) delete all employees whose performance_score is below 3.

Q17. What is a projection in MongoDB? Write a query to retrieve only the 'name' and 'salary' fields (excluding _id) for all employees in the 'HR' department.

Q18. Explain the difference between find() and findOne() in MongoDB. Write one example of each on an 'orders' collection.

Q19. Explain the use of $and, $or, and $not operators in MongoDB with a short example query each.

Q20. Explain MongoDB's replication briefly. How does a replica set ensure high availability when the primary node fails?

Q1. NoSQL Database Types and System Design
Describe in detail the four types of NoSQL databases — key-value, document, column-family, and graph stores. For each type, explain its internal data model, name one widely used database product, and give one real-world use case. Then, consider a healthcare startup building a patient management platform that must handle: (i) patient session tokens, (ii) medical records with varying fields per patient, (iii) appointment histories, and (iv) referral chains between doctors. Recommend the most suitable NoSQL type for each data category and justify your choices with reference to data access patterns.

Q2. CAP Theorem and SQL vs NoSQL vs NewSQL Comparison
Explain the CAP theorem in detail and draw a clearly labelled CAP triangle. Position MongoDB, Cassandra, HBase, and Redis on this triangle, stating which guarantee each sacrifices and why that trade-off suits its primary use case. Then prepare a structured six-dimension comparison of SQL (e.g., PostgreSQL), NoSQL (e.g., MongoDB), and NewSQL (e.g., CockroachDB) covering: (i) data model, (ii) schema flexibility, (iii) scalability model, (iv) consistency guarantee, (v) query language, and (vi) best-fit use case.

Q3. BASE Properties and Sharding Calculations
Define the three BASE properties (Basically Available, Soft State, Eventually Consistent) of NoSQL systems and contrast each with the corresponding ACID property using a concrete, realistic example. Identify at least two application domains where eventual consistency is fully acceptable. Then, consider a distributed key-value store holding 1.2 billion records sharded by consistent hashing across 12 nodes. Calculate: (i) the average number of records per shard, (ii) the expected number of node lookups for a single point-query, (iii) the expected number of shards that must be queried for a range query covering 8.33% of the total key space. Show all working clearly.

Q4. NoSQL in Modern Industry Use Cases
Discuss five major advantages of NoSQL databases over traditional RDBMS — flexible schema, horizontal scaling, high availability, polyglot persistence, and lower infrastructure cost. For each advantage, provide a specific real-world scenario where it is critical. Then explain how NoSQL databases are used in the following three industries: (i) e-commerce (e.g., Flipkart/Amazon) — describe the NoSQL type used and the specific data management problem it resolves; (ii) media streaming (e.g., Hotstar/Netflix) — explain how NoSQL handles personalisation and content metadata at scale; (iii) food-delivery platforms (e.g., Swiggy/Zomato) — describe how real-time location and order-state data is managed using NoSQL.

Q5. NewSQL Architecture and Workload Comparison
Explain NewSQL systems in depth. Describe how they achieve ACID compliance while enabling horizontal scalability, focusing on how distributed consensus mechanisms (Paxos or Raft) are used to maintain consistency in systems like CockroachDB or Google Spanner. Then, a global payments company processes 600,000 micro-transactions per second with strict ACID requirements and must scale across data centres in three continents. Compare SQL (PostgreSQL), NoSQL (Apache Cassandra), and NewSQL (CockroachDB) for this specific workload across the dimensions of consistency, throughput, and global distribution. Clearly recommend one option with a structured justification.

Q6. MongoDB BSON Data Types and Schema Design
List and explain all BSON data types supported by MongoDB, providing a concrete example value for each: String, Int32, Int64, Double, Boolean, Date, ObjectId, Array, Embedded Document, Null, Binary, Timestamp, and Decimal128. Then, design a MongoDB document schema for a university course registration system. The schema must capture: a Student document (with personal details, enrolled courses as an array where each entry contains course code, course name, instructor name, and grade), and an instructor reference using ObjectId. Write two fully formed sample documents for two different students, and explain your choice of data types and your decision to embed course details rather than reference them in a separate collection.

Q7. MongoDB Query Language — CRUD and Aggregation
Write MongoDB queries for the following on a 'products' collection (fields: name, price, category, stock, ratings[]): (i) Find all products in category 'Clothing' with price between 300 and 1500. (ii) Find all products where stock is 0 and update their status field to 'Unavailable'. (iii) Delete all products whose ratings array has an average below 2.5. (iv) Retrieve the top 5 most expensive products displaying only name and price. Then, write a complete MongoDB aggregation pipeline on a 'transactions' collection (fields: product_id, category, quantity, amount, date) to: (i) filter only transactions from the calendar year 2025; (ii) group by category computing total_revenue and total_quantity_sold; (iii) sort results by total_revenue in descending order; (iv) limit output to the top 4 categories.

Q8. MongoDB Schema Design — Embedding vs Referencing
Explain the embedding vs referencing design philosophies in MongoDB schema design. Consider an online learning platform with Instructors and Courses. Each course has one instructor and multiple enrolled students (each with enrolment date and completion status). Design two complete schema versions: (i) a fully embedded design, and (ii) a reference-based design. For each version, write at least one representative sample document. Then compare both designs on read performance, write performance, data duplication, and consistency risk, and state which design you would recommend for a platform with 50,000 active students and 200 courses, justifying your answer.

Q9. MongoDB Indexing and Aggregation Pipeline Stages
Explain MongoDB indexing in depth. For each of the following index types, write the createIndex() command and describe the use case it best serves: single-field index, compound index, multikey index (for array fields), text index, and TTL (Time-To-Live) index. Then explain the following MongoDB aggregation pipeline stages using a single 'orders' collection (fields: order_id, customer_id, items[], total_amount, status, order_date): $match, $group, $project, $sort, $limit, $unwind, and $lookup. For each stage, write a short but complete example pipeline snippet and explain what it produces.

Q10. MongoDB Replication Architecture and Availability
Explain MongoDB replication using replica sets in detail. Draw and describe the architecture showing the roles of the primary node, secondary nodes, and arbiter node. Walk through what happens internally during: (i) a normal write operation — describe the oplog propagation process; (ii) a primary node failure — describe how the election protocol chooses a new primary and how clients reconnect; (iii) a secondary node failure — describe the impact on reads and overall cluster health. Explain how the replica set configuration ensures high availability and zero data loss in the event of a single-node failure, and state the minimum recommended replica set size for a production deployment.

Q11. R Control Statements with Working Code
Explain all control flow statements in R with working code snippets and their expected outputs: if, if-else, if-else-if-else chain, for loop, while loop, repeat loop, break, and next. Clearly highlight the practical difference between break and next with a common loop example. Then write a complete, self-contained R program that: (i) creates a vector of 30 random integers between 1 and 150 using sample(); (ii) uses a for loop to separately count even and odd numbers; (iii) computes the mean of the even subset and the mean of the odd subset; (iv) prints a formatted summary showing even count, odd count, even mean, and odd mean. Include the full code and describe the expected form of the output.

Q12. R Matrix Operations and Linear Algebra
Write complete R code to perform the following matrix operations and show the output for each step: (i) Create a 4×4 matrix M using values 1:16 filled column-wise. (ii) Compute row sums and column sums using apply() with MARGIN = 1 and MARGIN = 2. (iii) Compute the matrix inverse of M using solve() — if M is singular, use A = matrix(c(2,1,5,3), nrow=2) and B = matrix(c(1,-1,-5,2), nrow=2) instead; compute A + B, A %*% B, t(A), det(A), and solve(A). (iv) Verify that A %*% solve(A) produces an identity matrix. (v) Explain what a singular matrix is and why solve() would fail for such a matrix.

Q13. R Apply Family Functions and CSV File I/O
Explain the four main functions of the apply family in R with detailed, working examples: (i) apply() — compute the column-wise standard deviation of a 6×4 matrix filled with random normal values; (ii) lapply() — compute the factorial of each element in list(4, 6, 8, 10); (iii) sapply() — repeat the same computation as lapply() but return a named numeric vector; (iv) tapply() — compute the mean exam score grouped by department from a data frame with columns Name, Dept, and Score. Then write R

code to read a CSV file 'exam_results.csv' (columns: Month, Centre, Score). Perform: (i) display the first and last 6 rows; (ii) compute total score per Centre using tapply(); (iii) identify the Centre with the highest total score; (iv) write a summary report to 'report.txt' using write(); (v) wrap the file-read operation in tryCatch() to handle the case where the file does not exist.

Q14. R Data Frames, Factors, and Visualisation

Create an R data frame 'students' with 6 records containing columns: Name (character), Age (integer), Score (numeric), Dept (factor with levels CS / IT / ECE). Write R code to: (i) display the structure using str() and summary statistics using summary(); (ii) filter students with Score >= 65; (iii) sort the data frame by Score in descending order; (iv) add a new column 'Grade' mapping Score >= 80 to 'Distinction', 60-79 to 'Pass', and below 60 to 'Fail'. Then, given a vector: grades = c('B','A','C','A','B','A','C','B','A','D','B','C'), convert it to an ordered factor with levels D < C < B < A, display the levels and nlevels, create a frequency table using table(), and plot a colour-coded bar chart with appropriate title and axis labels.

Q15. R User-Defined Functions and Lists

Write two user-defined functions in R with complete code and demonstrated output: (i) A recursive function 'compute_gcd(a, b)' that finds the Greatest Common Divisor using Euclid's algorithm. Call it for the pairs (72, 48) and (120, 85) and display the results. (ii) A function 'describe_vector(v)' that accepts a numeric vector and returns a named list containing: mean, median, variance, standard deviation, minimum, and maximum. Test it on v = c(15, 38, 9, 27, 64, 43, 82, 11, 55). Then explain R lists in detail. Create a list 'employee' with: name='Priya', id=204, salary=68000.75, skills=c('R','Python','Tableau'), active=TRUE. Demonstrate accessing each element using [[]] and $ notation, adding a new field 'department'='Analytics', and converting the flat scalar elements (excluding the skills vector) into a data frame.